

Experiment No. 5

□ **Aim:** Hands on Solidity Programming Assignments for creating Smart Contracts

□ **Theory:**

1. Primitive Data Types, Variables, Functions – pure, view

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- **uint / int:** unsigned and signed integers of different sizes (e.g., uint256, int128).
- **bool:** represents logical values (true or false).
- **address:** holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- **bytes / string:** store binary data or textual data.

Variables in Solidity can be **state variables** (stored on the blockchain permanently), **local variables** (temporary, created during function execution), or **global variables** (special predefined variables such as msg.sender, msg.value, and block.timestamp).

Functions allow execution of contract logic. Special types of functions include:

- **pure:** cannot read or modify blockchain state; they work only with inputs and internal computations.
- **view:** can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

2. Inputs and Outputs to Functions

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

3. Visibility, Modifiers and Constructors

- **Function Visibility** defines who can access a function:

- o public: available both inside and outside the contract.
- o private: only accessible within the same contract.
- o internal: accessible within the contract and its child contracts.
- o external: can be called only by external accounts or other contracts.
- **Modifiers** are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).
- **Constructors** are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

3. Control Flow: if-else, loops

Control flow in Solidity is similar to traditional programming languages:

- **if-else** allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.
- **Loops** (for, while, do-while) enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

5. Data Structures: Arrays, Mappings, Structs, Enums

- **Arrays:** Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.
- **Mappings:** Key-value pairs that allow quick lookups. Example: mapping(address => uint) for storing balances. Unlike arrays, mappings do not support iteration.
- **Structs:** Allow grouping of related properties into a single data type, such as creating a struct Player {string name; uint score;}.
- **Enums:** Used to define a set of predefined constants, making code more readable. Example: enum Status { Pending, Active, Closed }.

6. Data Locations

Solidity uses three primary data locations for storing variables:

- **storage:** Data stored permanently on the blockchain. Examples: state variables.
- **memory:** Temporary data storage that exists only while a function is executing.

Used for local variables and function inputs.

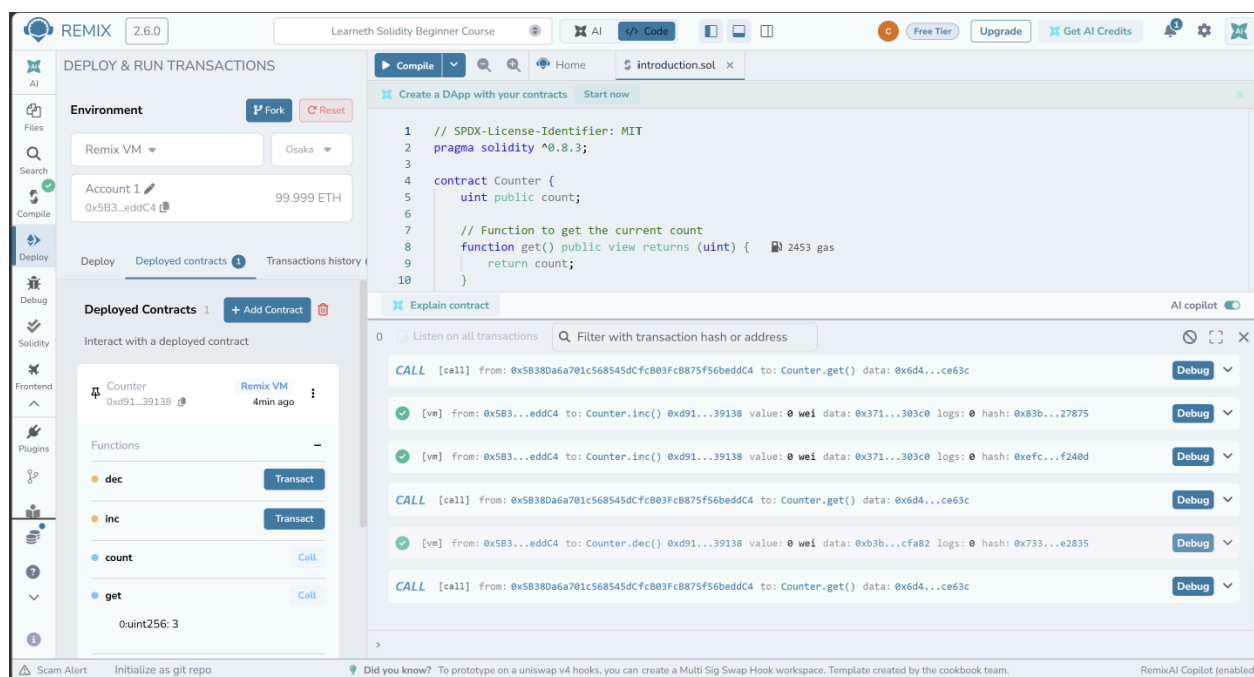
- **calldata:** A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory. Understanding data locations is essential, as they directly impact gas costs and performance.

7. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

- **Ether and Wei:** Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether = 10^{18} Wei). This ensures high precision in financial transactions.
- **Gas and Gas Price:** Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.
- **Sending Transactions:** Transactions are used for transferring Ether or interacting with contracts. Functions like `transfer()` and `send()` are commonly used, while `call()` provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

? Implementation:

- Tutorial no. 1 – Compile the code



- Tutorial no. 2

REMIX 2.6.0 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

2. Basic Syntax 2 / 19

We also define the **visibility** of the variable, which specifies from where you can access it. In this case, it's a **public** variable that you can access from inside and outside the contract.

Don't worry if you didn't understand some concepts like **visibility**, **data types**, or **state variables**. We will look into them in the following sections.

To help you understand the code, we will link in all following sections to video tutorials from the **creator** of the Solidity by Example contracts.

Watch a video tutorial on Basic Syntax.

★ **Assignment**

1. Delete the HelloWorld contract and its content.
2. Create a new contract named "MyContract".
3. The contract should have a public state variable called "name" of the type string.
4. Assign the value "Alice" to your new variable.

Check Answer Show answer

Errors

Error: Error from IDE : [TIMEOUT] Timeout for call testFromPath from LearnEth

Scam Alert Initialize as git repo Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. RemixAI Copilot (enabled)

AI

Code

```
1 // SPDX-License-Identifier: MIT
2 // compiler version must be greater than or equal to 0.8.3 and less than 0.9.0
3 pragma solidity ^0.8.3;
4
5 contract MyContract {
6     string public name = "Alice";
7 }
```

Compile

Home

introduction.sol basicSyntax.sol basicSyntax_answer.sol

Create a DApp with your contracts Start now

AI copilot

0 Listen on all transactions Filter with transaction hash or address

CALL [call] from: 0x58380a6a701c56854dCfcB03Fc8875f56beddC4 to: Counter.get() data: 0x6d4...ce63c Debug

[vm] from: 0x583...eddC4 to: Counter.inc() 0xd91...39138 value: 0 wei data: 0x371...303c0 logs: 0 Debug

[vm] from: 0x583...eddC4 to: Counter.inc() 0xd91...39138 value: 0 wei data: 0x371...303c0 logs: 0 Debug

CALL [call] from: 0x58380a6a701c56854dCfcB03Fc8875f56beddC4 to: Counter.get() data: 0x6d4...ce63c Debug

[vm] from: 0x583...eddC4 to: Counter.dec() 0xd91...39138 value: 0 wei data: 0xb3b...cfa82 logs: 0 Debug

CALL [call] from: 0x58380a6a701c56854dCfcB03Fc8875f56beddC4 to: Counter.get() data: 0x6d4...ce63c Debug

• Tutorial no. 3

REMIX 2.6.0 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

3. Primitive Data Types 3 / 19

You can learn more about these data types as well as **Fixed Point Numbers**, **Byte Arrays**, **Strings**, and more in the **Solidity documentation**.

Later in the course, we will look at data structures like **Mappings**, **Arrays**, **Enums**, and **Structs**.

Watch a video tutorial on Primitive Data Types.

★ **Assignment**

1. Create a new variable **newAddr** that is a **public address** and give it a value that is not the same as the available variable **addr**.
2. Create a **public** variable called **neg** that is a negative number, decide upon the type.
3. Create a new variable, **newU** that has the smallest **uint** size type and the smallest **uint** value and is **public**.

Tip: Look at the other address in the contract or search the internet for an Ethereum address.

Check Answer Show answer

Errors

Error: Error from IDE : [TIMEOUT] Timeout for call testFromPath from LearnEth

Scam Alert Initialize as git repo Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. RemixAI Copilot (enabled)

AI

Code

```
20 Negative numbers are allowed for int types.
21 Like uint, different ranges are available from int8 to int256
22 */
23 int8 public i8 = -1;
24 int public i256 = 456;
25 int public i = -123; // int is same as int256
26
27 address public addr = 0xCa35b7d915458EF540aDe068dFe2F44E8fa733c;
28
29 // Default values
30 // Unassigned variables have a default value
31 bool public defaultBool = false;
32 uint public defaultUint = 0;
33 int public defaultInt = 0;
34 address public defaultAddr; // 0x0000000000000000000000000000000000000000000000000000000000000000;
35
36 address public newAddr = 0x0000000000000000000000000000000000000000000000000000000000000000;
37 int public neg = -10;
38 uint public newU = 0;
39 }
```

Compile

Home

cSyntax_answer.sol primitiveDataTypes.sol basicSyntax.sol variables.sol

Create a DApp with your contracts Start now

AI copilot

0 Listen on all transactions Filter with transaction hash or address

- Tutorial no. 4

REMIX 2.6.0 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

4. Variables 4 / 19

need to be declared but can be accessed from within your contract. Global variables are used to retrieve information about the blockchain, particular addresses, contracts, and transactions.

In this example, we use `block.timestamp` (line 14) to get a Unix timestamp of when the current block was generated and `msg.sender` (line 15) to get the caller of the contract function's address.

A list of all Global Variables is available in the [Solidity documentation](#).

Watch video tutorials on [State Variables](#), [Local Variables](#), and [Global Variables](#).

Assignment

1. Create a new public state variable called `blockNumber`.
2. Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

Check Answer Show answer

Errors

Error: Error from IDE : [TIMEOUT] Timeout for call testFromPath from LearnEth

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Variables {
5     // State variables are stored on the blockchain.
6     string public text = "Hello";
7     uint public num = 123;
8
9     // Stores the current block number
10    uint public blockNumber;
11
12    function doSomething() public { 22334 gas
13        // Local variable
14        uint i = 456;
15
16        // Assign current block number to state variable
17        blockNumber = block.number;
18
19        // Global variables
20        uint timestamp = block.timestamp; // Current block timestamp
21        address sender = msg.sender; // Address of the caller
22    }
23 }

```

Explain contract AI copilot

0 ☐ Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. RemixAI Copilot (enabled)

- Tutorial no. 5

REMIX 2.6.0 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

5.1 Functions - Reading and Writing to a State Variable 5 / 19

If the function takes inputs like our `set` function (line 9), you must specify the parameter types and names. A common convention is to use an underscore as a prefix for the parameter name to distinguish them from state variables.

You can then set the visibility of a function and declare them `view` or `pure` as we do for the `get` function if they don't modify the state. Our `get` function also returns values, so we have to specify the return types. In this case, it's a `uint` since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections.

Watch a video tutorial on [Functions](#).

Assignment

1. Create a public state variable called `b` that is of type `bool` and initialize it to `true`.
2. Create a public function called `get_b` that returns the value of `b`.

Check Answer Show answer

Errors

Error: Error from IDE : [TIMEOUT] Timeout for call testFromPath from LearnEth

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract SimpleStorage {
5     // State variable to store a number
6     uint public num;
7
8     // Boolean state variable
9     bool public b = true;
10
11    // You need to send a transaction to write to a state variable.
12    function set(uint _num) public { 22536 gas
13        num = _num;
14    }
15
16    // You can read from a state variable without sending a transaction.
17    function get() public view returns (uint) { 2475 gas
18        return num;
19    }
20
21    // Returns the value of b
22    function get_b() public view returns (bool) { 2539 gas
23        return b;
24    }
25 }

```

Explain contract AI copilot

0 ☐ Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. RemixAI Copilot (enabled)

• Tutorial no. 6

REMIX 2.6.0 | Learneth Solidity Beginner Course

LEARNETH | Tutorials list | Syllabus

5.2 Functions - View and Pure | 6 / 19

run contract initialization code. The constructor can have parameters and is especially useful when you don't know certain initialization values before the deployment of the contract.

You declare a constructor using the `constructor` keyword. The constructor in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

Watch a video tutorial on Function Modifiers.

Assignment

1. Create a new function, `increaseX` in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.

2. Make sure that `x` can only be increased.

3. The body of the function `increaseX` should be empty.

Tip: Use modifiers.

Check Answer | **Show answer**

Errors

Error from IDE : [TIMEOUT] Timeout for call testFromPath from LearnEth

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ViewAndPure {
5     uint public x = 1;
6
7     // Promise not to modify the state.
8     function addToX(uint y) public view returns (uint) { infinite gas
9         return x + y;
10    }
11
12    // Promise not to modify or read from the state.
13    function add(uint i, uint j) public pure returns (uint) { infinite gas
14        return i + j;
15    }
16
17    function addToX2(uint y) public { infinite gas
18        x=x+y;
19    }
20 }

```

Explain contract | AI copilot

0 | Listen on all transactions | Filter with transaction hash or address

Scam Alert | Initialize as git repo | Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. | RemixAI Copilot (enabled)

• Tutorial no. 7

REMIX 2.6.0 | Learneth Solidity Beginner Course

LEARNETH | Tutorials list | Syllabus

5.3 Functions - Modifiers and Constructors | 7 / 19

run contract initialization code. The constructor can have parameters and is especially useful when you don't know certain initialization values before the deployment of the contract.

You declare a constructor using the `constructor` keyword. The constructor in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

Watch a video tutorial on Function Modifiers.

Assignment

1. Create a new function, `increaseX` in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.

2. Make sure that `x` can only be increased.

3. The body of the function `increaseX` should be empty.

Tip: Use modifiers.

Check Answer | **Show answer**

Errors

Error from IDE : [TIMEOUT] Timeout for call testFromPath from LearnEth

```

35     locked = false;
36
37 }
38
39 function decrement(uint i) public noReentrancy { infinite gas
40     x -= i;
41
42     if (i > 1) {
43         decrement(i - 1);
44     }
45 }
46
47 // Modifier increases x by the given amount
48 modifier increaseOnly(uint i) {
49     x += i;
50     _;
51 }
52
53 // Assignment
54 function increaseX(uint i) public increaseOnly(i) { infinite gas
55 }
56 }

```

Explain contract | AI copilot

0 | Listen on all transactions | Filter with transaction hash or address

Scam Alert | Initialize as git repo | Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. | RemixAI Copilot (enabled)

• Tutorial no. 8

REMIX 2.6.0 | Learneth Solidity Beginner Course

LEARNETH | Tutorials list | Syllabus

5.4 Functions - Inputs and Outputs (8 / 19)

There are a few restrictions and best practices for the input and output parameters of contract functions.

"[Mappings] cannot be used as parameters or return parameters of contract functions that are publicly visible." From the [Solidity documentation](#).

Arrays can be used as parameters, as shown in the function `arrayInput` (line 71). Arrays can also be used as return parameters as shown in the function `arrayOutput` (line 76).

You have to be cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.

Watch a video tutorial on [Function Outputs](#).

Assignment

Create a new function called `returnTwo` that returns the values `-2` and `true` without using a return statement.

[Check Answer](#) [Show answer](#)

Errors

Error: Error from IDE : [TIMEOUT] Timeout for call testFromPath from LearnEth

Code Editor: `inputsAndOutputs.sol`

```

65     return (i, b, j, x, y);
66 }
67
68 // Cannot use map for neither input nor output
69
70 // Can use array for input
71 function arrayInput(uint[] memory _arr) public { infinite gas
72
73 // Can use array for output
74 uint[] public arr;
75
76 function arrayOutput() public view returns (uint[] memory) { infinite gas
77     return arr;
78 }
79
80 function returnTwo() public pure returns(int p,bool q){ 472 gas
81     p=-2;
82     q=true;
83 }
84 }

```

AI Copilot: Explain contract | Listen on all transactions | Filter with transaction hash or address

Scam Alert | Initialize as git repo | Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. | RemixAI Copilot (enabled)

• Tutorial no. 9

REMIX 2.6.0 | Learneth Solidity Beginner Course

LEARNETH | Tutorials list | Syllabus

6. Visibility (9 / 19)

State variables can not be `external`.

In this example, we have two contracts, the `Base` contract (line 4) and the `Child` contract (line 55) which inherits the functions and state variables from the `Base` contract.

When you uncomment the `testPrivateFunc` (lines 58-60) you get an error because the child contract doesn't have access to the private function `privateFunc` from the `Base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `privateFunc` and `internalFunc` directly. You will only be able to call them via `testPrivateFunc` and `testInternalFunc`.

Watch a video tutorial on [Visibility](#).

Assignment

Create a new function in the `Child` contract called `testInternalVar` that returns the values of all state variables from the `Base` contract that are possible to return.

[Check Answer](#) [Show answer](#)

Errors

Error: Error from IDE : [TIMEOUT] Timeout for call testFromPath from LearnEth

Code Editor: `visibility.sol`

```

26     return "external function called";
27 }
28
29 // State variables
30 string private privateVar = "my private variable";
31 string internal internalVar = "my internal variable";
32 string public publicVar = "my public variable";
33 }
34
35 contract Child is Base {
36
37     function testInternalFunc() public pure override returns (string memory) { infinite gas
38         return internalFunc();
39     }
40
41 // Assignment
42 function testInternalVar() infinite gas
43     public
44     view
45     returns (string memory, string memory)
46 {
47     return (internalVar, publicVar);
48 }
49 }

```

AI Copilot: Explain contract | Listen on all transactions | Filter with transaction hash or address

Scam Alert | Initialize as git repo | Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. | RemixAI Copilot (enabled)

• Tutorial no. 10

LEARNETH 2.6.0 Learneth Solidity Beginner Course

Tutorials list 7.1 Control Flow - If/Else 10 / 19

Assignment

Create a new function called `evenCheck` in the `ifElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check Answer **Show answer**

Errors

Error: Error from IDE: [TIMEOUT] Timeout for call testFromPath from LearnEth

```

12 }
13 }
14
15 function ternary(uint _x) public pure returns (uint) {
16     // if (_x < 10) {
17     //     return 1;
18     // }
19     // return 2;
20
21     // shorthand way to write if / else statement
22     return _x < 10 ? 1 : 2;
23 }
24
25 function evenCheck(uint p) public pure returns (bool) {
26     if (p % 2 == 0) {
27         return true;
28     } else {
29         return false;
30     }
31 }

```

AI copilot: ☒ Listen on all transactions

Scam Alert Initialize as git repo Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. RemixAI Copilot (enabled)

• Tutorial no. 11

LEARNETH 2.6.0 Learneth Solidity Beginner Course

Tutorials list 7.2 Control Flow - Loops 11 / 19

Assignment

1. Create a public `uint` state variable called `count` in the `Loop` contract.

2. At the end of the for loop, increment the count variable by 1.

3. Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

Check Answer **Show answer**

Errors

Error: Error from IDE: [TIMEOUT] Timeout for call testFromPath from LearnEth

```

4 contract Loop {
5     uint public count;
6
7     function loop() public {
8         // for loop
9         for (uint i = 0; i < 10; i++) {
10             if (i == 3) {
11                 // Skip to next iteration with continue
12                 continue;
13             }
14
15             if (i == 5) {
16                 // Exit loop with break
17                 break;
18             }
19
20             count++;
21         }
22
23         // while loop
24         uint j;
25         while (j < 5) {
26             j++;
27             count++;
28         }
29     }
30 }

```

AI copilot: ☒ Listen on all transactions

Scam Alert Initialize as git repo Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. RemixAI Copilot (enabled)

• Tutorial no. 12

LEARNETH

8.1 Data Structures - Arrays

array (line 42). When we remove an element with the `delete` operator all other elements stay the same, which means that the length of the array will stay the same. This will create a gap in our array. If the order of the array is not important, then we can move the last element of the array to the place of the deleted element (line 46), or use a mapping. A mapping might be a better choice if we plan to remove elements in our data structure.

Array length

Using the length member, we can read the number of elements that are stored in an array (line 35).

[Watch a video tutorial on Arrays.](#)

★ **Assignment**

1. Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.
2. Change the `getArr()` function to return the value of `arr3`.

[Check Answer](#) [Show answer](#)

Errors

Error from IDE : [TIMEOUT] Timeout for call testFromPath from LearnEth

```

1 contract Array {
2   uint[] public arr;
3   uint[] public arr2 = [1, 2, 3];
4
5   uint[3] public arr3 = [1, 2, 3];
6
7   uint[10] public myFixedSizeArr;
8
9   function get(uint i) public view returns (uint) {
10    return arr[i];
11  }
12
13  function getArr() public view returns (uint[3] memory) {
14    return arr3;
15  }
16
17  function push(uint i) public {
18    arr.push(i);
19  }
20
21  function pop() public {
22    arr.pop();
23  }
24
25  function getLength() public view returns (uint) {
26    return arr.length;
27  }

```

Scam Alert Initialize as git repo Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. RemixAI Copilot (enabled)

• Tutorial no. 13

LEARNETH

8.2 Data Structures - Mappings

We set a new value for a key by providing the mapping's name and key in brackets and assigning it a new value (line 16).

Removing values

We can use the delete operator to delete a value associated with a key, which will set it to the default value of 0. As we have seen in the arrays section.

[Watch a video tutorial on Mappings.](#)

★ **Assignment**

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping balances.
3. Change the function `set` to create a new entry to the balances mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

[Check Answer](#) [Show answer](#)

Errors

Error from IDE : [TIMEOUT] Timeout for call testFromPath from LearnEth

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Mapping {
5   // Mapping from address to uint
6   mapping(address => uint) public balances;
7
8   function get(address _addr) public view returns (uint) {
9     return balances[_addr];
10  }
11
12  function set(address _addr, uint _value) public {
13    balances[_addr] = _value;
14  }
15
16  function remove(address _addr) public {
17    delete balances[_addr];
18  }
19 }
20
21 contract NestedMapping {
22   mapping(address => mapping(uint => bool)) public nested;
23
24   function get(address _addr1, uint _i) public view returns (bool) {
25     return nested[_addr1][_i];
26   }
27
28   function set(address _addr1, uint _i, bool _value) public {

```

Scam Alert Initialize as git repo Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. RemixAI Copilot (enabled)

• Tutorial no. 14

The screenshot shows the Remix IDE interface. On the left, the 'Tutorials list' sidebar is open, displaying '8.3 Data Structures - Structs' (14 / 19). The tutorial content includes sections for 'Accessing structs' and 'Updating structs', followed by an 'Assignment' that asks the user to create a function named 'remove' that takes a 'uint' as a parameter and deletes a struct member from the 'todos' mapping. The main editor area shows a Solidity file named 'structs.sol' with the following code:

```
33     Todo storage todo = todos[_index];
34     return (todo.text, todo.completed);
35 }
36
37 function update(uint _index, string memory _text) public {
38     Todo storage todo = todos[_index];
39     todo.text = _text;
40 }
41
42 function toggleCompleted(uint _index) public {
43     Todo storage todo = todos[_index];
44     todo.completed = !todo.completed;
45 }
46
47 function remove(uint _index) public {
48     todos[_index] = todos[todos.length - 1];
49     todos.pop();
50 }
51 }
```

The bottom status bar indicates 'Scam Alert', 'Initialize as git repo', and 'Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team.' The 'RemixAI Copilot (enabled)' button is visible in the bottom right corner.

• Tutorial no. 15

The screenshot shows the Remix IDE interface. On the left, the 'Tutorials list' sidebar is open, displaying '8.4 Data Structures - Enums' (15 / 19). The tutorial content includes an 'Assignment' with three steps: 1. Define an enum type called 'Size' with members 'S', 'M', and 'L'. 2. Initialize the variable 'sizes' of the enum type 'Size'. 3. Create a getter function 'getSize()' that returns the value of the variable 'sizes'. The main editor area shows a Solidity file named 'enums.sol' with the following code:

```
1
2 // SPDX-License-Identifier: MIT
3 pragma solidity ^0.8.3;
4
5 contract Enum {
6     // Enum representing sizes
7     enum Size {
8         S,
9         M,
10        L
11    }
12
13    // Default value is the first element, S
14    Size public sizes;
15
16    // Getter function
17    function getSize() public view returns (Size) {
18        return sizes;
19    }
20 }
21
22
```

The bottom status bar indicates 'Scam Alert', 'Initialize as git repo', and 'Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team.' The 'RemixAI Copilot (enabled)' button is visible in the bottom right corner.

• Tutorial no. 16

LEARNETH 2.6.0 Learneth Solidity Beginner Course

Tutorials list 9. Data Locations 16 / 19

2. Create a new struct `myMemStruct2` with the data location `memory` inside the `function f` and assign it the value of `myMemStruct`. Change the value of the `myMemStruct2` member `foo` to 1.
3. Create a new struct `myMemStruct3` with the data location `memory` inside the `function f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.
4. Let the function `f` return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Create a DApp with your contracts Start now

```

45 }
46
47 function _f( /* undefined gas
48     uint[] storage _arr,
49     mapping(uint => address) storage _map,
50     MyStruct storage _myStruct
51 ) internal {
52     // do something with storage variables
53 }
54
55 // You can return memory variables
56 function g(uint[] memory _arr) /* infinite gas
57     public
58     returns (uint[] memory)
59 {
60     return _arr;
61 }
62
63 function h(uint[] calldata _arr) external { /* 468 gas
64     // do something with calldata array
65 }
66
67
68

```

[Explain contract](#) AI copilot

Scam Alert Initialize as git repo Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. RemixAI Copilot (enabled)

• Tutorial no. 17

LEARNETH 2.6.0 Learneth Solidity Beginner Course

Tutorials list 10.1 Transactions - Ether and Wei 17 / 19

One `ether` is equal to 1000000000000000000 (10¹⁸) `wei` (line 11).

Watch a video tutorial on Ether and Wei.

Assignment

1. Create a `public uint` called `oneGwei` and set it to 1 `gwei`.
2. Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 `gwei` and 10⁹.

Tip: Look at how this is written for `gwei` and `ether` in the contract.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Create a DApp with your contracts Start now

```

1
2 // SPDX-License-Identifier: MIT
3 pragma solidity ^0.8.3;
4
5 contract EtherUnits {
6     uint public oneWei = 1 wei;
7     // 1 wei is equal to 1
8     bool public isOneWei = 1 wei == 1;
9
10    uint public oneEther = 1 ether;
11    // 1 ether is equal to 10^18 wei
12    bool public isOneEther = 1 ether == 1e18;
13
14    // 1 gwei is equal to 10^9 wei
15    uint public oneGwei = 1 gwei;
16
17    // Check if 1 gwei is equal to 10^9 wei
18    bool public isOneGwei = 1 gwei == 1e9;
19 }
20
21

```

[Explain contract](#) AI copilot

Scam Alert Initialize as git repo Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. RemixAI Copilot (enabled)

- Tutorial no. 18

LEARNETH 2.6.0 Learneth Solidity Beginner Course

Tutorials list 10.2 Transactions - Gas and Gas Price 18 / 19

Learn more about gas on ethereum.org.
Watch a video tutorial on Gas and Gas Price.

Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin *Gas Profiler* to check for the gas cost of transactions.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```

1
2 // SPDX-License-Identifier: MIT
3 pragma solidity ^0.8.3;
4
5 contract Gas {
6     uint public i = 0;
7
8     uint public cost = 170367;
9
10    function forever() public {
11        while (true) {
12            i += 1;
13        }
14    }
15 }
16
17

```

[Explain contract](#) AI copilot ☒

Scam Alert Initialize as git repo Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. RemixAI Copilot (enabled)

- Tutorial no. 19

LEARNETH 2.6.0 Learneth Solidity Beginner Course

Tutorials list 10.3 Transactions - Sending Ether 19 / 19

1. Create a contract called `Charity`.
2. Add a public state variable called `owner` of the type `address`.
3. Create a donate function that is public and payable without any parameters or function code.
4. Create a withdraw function that is public and sends the total balance of the contract to the `owner` address.

Tip: Test your contract by deploying it from one account and then sending Ether to it from another account. Then execute the withdraw function.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```

1
2 // SPDX-License-Identifier: MIT
3 pragma solidity ^0.8.3;
4
5 contract Charity {
6     // Address that can withdraw the Ether
7     address public owner;
8
9     // Set the owner when the contract is deployed
10    constructor() {
11        owner = msg.sender;
12    }
13
14    // Receive Ether
15    function donate() public payable {
16    }
17
18    // Withdraw the entire contract balance to the owner
19    function withdraw() public {
20        payable(owner).transfer(address(this).balance);
21    }
22 }
23

```

[Explain contract](#) AI copilot ☒

Scam Alert Initialize as git repo Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. RemixAI Copilot (enabled)

□ **Conclusion:**

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.